

Instruction Execution – Building a Data Path – Designing a Control Unit – Hardwired Control, Microprogrammed Control – Pipelining – Data Hazard – Control Hazards.

I. Instruction Execution

Steps in detail:

- ❖ All instructions start by using the program counter to supply the instruction address to the instruction memory.
- ❖ After the instruction is fetched, the register operands used by an instruction are specified by fields of that instruction.
- ❖ Once the register operands have been fetched, all the instruction classes, except jump, use the ALU after reading the registers.
 - Memory reference instructions (load or store) use the ALU for an address calculation.
 - Arithmetic Logical instructions use the ALU for the operation execution.
 - Branches use the ALU for comparison.
- ❖ The second input to the ALU can come from a register or the immediate field of the instruction.
- ❖ After using the ALU, the actions required to complete various instruction classes are not same.
 - If the operation is a memory reference instruction a load or store, the ALU result is used as an address to either store a value from the registers or load a value from memory into the registers. The result from the ALU or memory is written back into the register file.
 - If the instruction is an arithmetic-logical instruction, the result from the ALU must be written to a register.
 - Branches require the use of the ALU output to determine the next instruction address, which comes either from the ALU (where the PC and branch off set are summed) or from an adder that increments the current PC by 4.

Main 5 steps

1. Fetch an instruction and increment the program counter.
2. Decode the instruction and read registers from the register file.
3. Perform an ALU operation.
4. Read or write memory data if the instruction involves a memory operand.
5. Write the result into the destination register, if needed.

Load Instruction

Eg. Load R5, X(R7)

Steps are as follows:

1. Fetch the instruction from the memory.
2. Increment the program counter.

3. Decode the instruction to determine the operation to be performed.
4. Read register R7.
5. Add the immediate value X to the contents of R7.
6. Use the sum $X + [R7]$ as the effective address of the source operand, and read the contents of that location in the memory.
7. Load the data received from the memory into the destination register, R5.
- ❖ Depending on how the hardware is organized, some of these actions can be performed at the same time.

Arithmetic and Logic Instruction

- There are either two source registers, or a source register and an immediate source operand.
- No access to memory operands is required.

Eg. Add R3, R4, R5

Steps as follows

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read registers R4 and R5.
3. Compute the sum $[R4] + [R5]$.
4. No action.
5. Load the result into the destination register, R3.

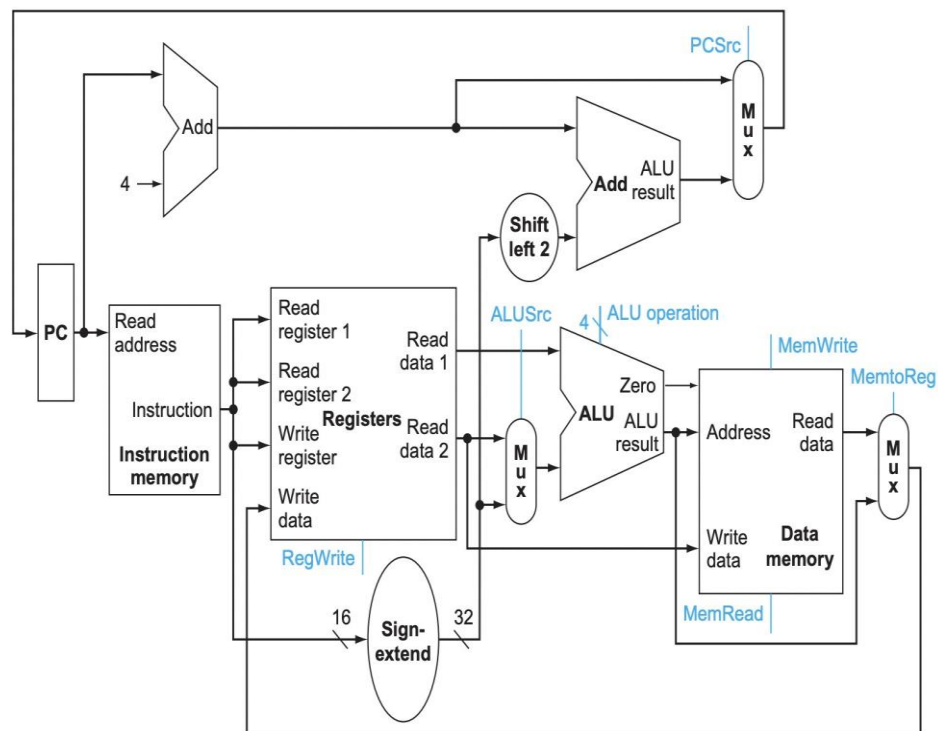
Store Instruction

Store R6, X(R8)

Steps as follows:

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read registers R6 and R8.
3. Compute the effective address $X + [R8]$.
4. Store the contents of register R6 into memory location $X + [R8]$.
5. No action.

II. Building a Datapath - Diagram is Mandatory



Datapath

- A **datapath** is a collection of functional units such as arithmetic logic units or multipliers that perform data processing operations, registers, and buses. Along with the control unit it composes the central processing unit (CPU).
- A larger datapath can be made by joining more than one datapaths using multiplexers.

1. Program Counter(PC)

A program counter is a **register in a computer processor that contains the address (location) of the instruction being executed at the current time**. As each instruction gets fetched, the program counter increases its stored value by 1.

2. Adder

Used to increment the PC to the address of the next instruction. It is built from the ALU.

3. Instruction Memory

- A memory unit to store the instructions of a program and supply instructions given an address.

Components 1 + 2 + 3 performs : form a datapath that fetches instructions and increments the PC to obtain the address of the next sequential instruction.

4. Registers

- ❖ The processor's 32 general-purpose registers are stored in a structure called a register file. A register file is a collection of registers in which any register can be read or written by specifying the number of the register in the file.
- ❖ The register file contains the register state of the computer.
- ❖ An ALU is used to operate on the values read from the registers.

5. Processing of R- format instruction in ALU:

add \$t1,\$t2,\$t3,

Example

- ❖ R-format instructions have three register operands, so we will need to read two data words from the register file and write one data word into the register file for each instruction.
- ❖ For each data word to be read from the registers, we need an input to the register file that specifies the register number to be read and an output from the register file that will carry the value that has been read from the registers.
- ❖ The two values read are added using an **ALU**.
- ❖ To write a data word, we will need two inputs: one to specify the register number to be written and one to supply the data to be written into the register.

6. Processing of Load/Store Instruction:

Example lw \$t1,offset_value(\$t2)

sw \$t1,offset_value (\$t2)

1. Sign Extend- Convert the 16-bit offset field in the instruction to a 32-bit signed value.
2. Data Memory - The memory unit is a state element with inputs for the address and the write data, and a single output for the read result. There are separate read and write controls, although only one of these may be asserted on any given clock.

7. Processing of Jump Instructions

Eg. beq \$t1,\$t2,offset

Explanation of example :

The **beq** instruction has three operands, two registers that are compared for equality.

If contents of t1 == contents of t2 — Compute target using offset and take branch. (ALU used to check equality- If zero flag is set means t1==t2)

Else - Proceed with next instruction

1. **Separate adder** - Used for computing the branch target address.
2. **Shift left** - Used to add two zeroes to the low-order end of the sign-extended offset field.

Multiplexer - It is mainly used to select the circuit combination as per the nature of instruction.

Control Signals - I have not covered this in video. But you can read if required.

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

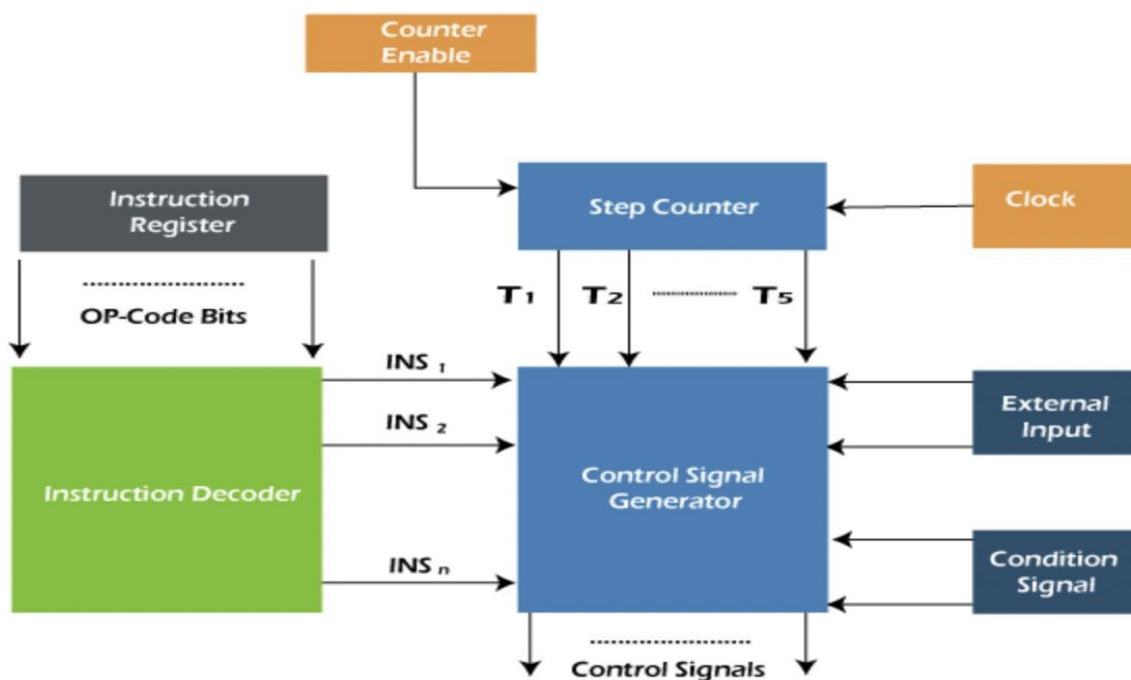
III. Control Unit

The setting of the control signals depends on:

- Contents of the step counter
- Contents of the instruction register
- The result of a computation or a comparison operation
- External input signals, such as interrupt requests

Hardwired Control Unit

- It is a method of generating control signals with the help of Finite State Machines (FSM). It's made in the form of a sequential logic circuit by physically connecting components such as flip-flops, gates, and drums that result in the finished circuit. As a result, it's known as a hardwired controller.



- **Instruction register** is a type of processor register used to contain an instruction that is currently in execution. It generates the OP-code bits respective of the operation as well as the addressing mode of operands.
- The instruction decoder decodes the opcode. Now on the basis of the addressing

mode of instruction and operation which exists in the instruction register, the

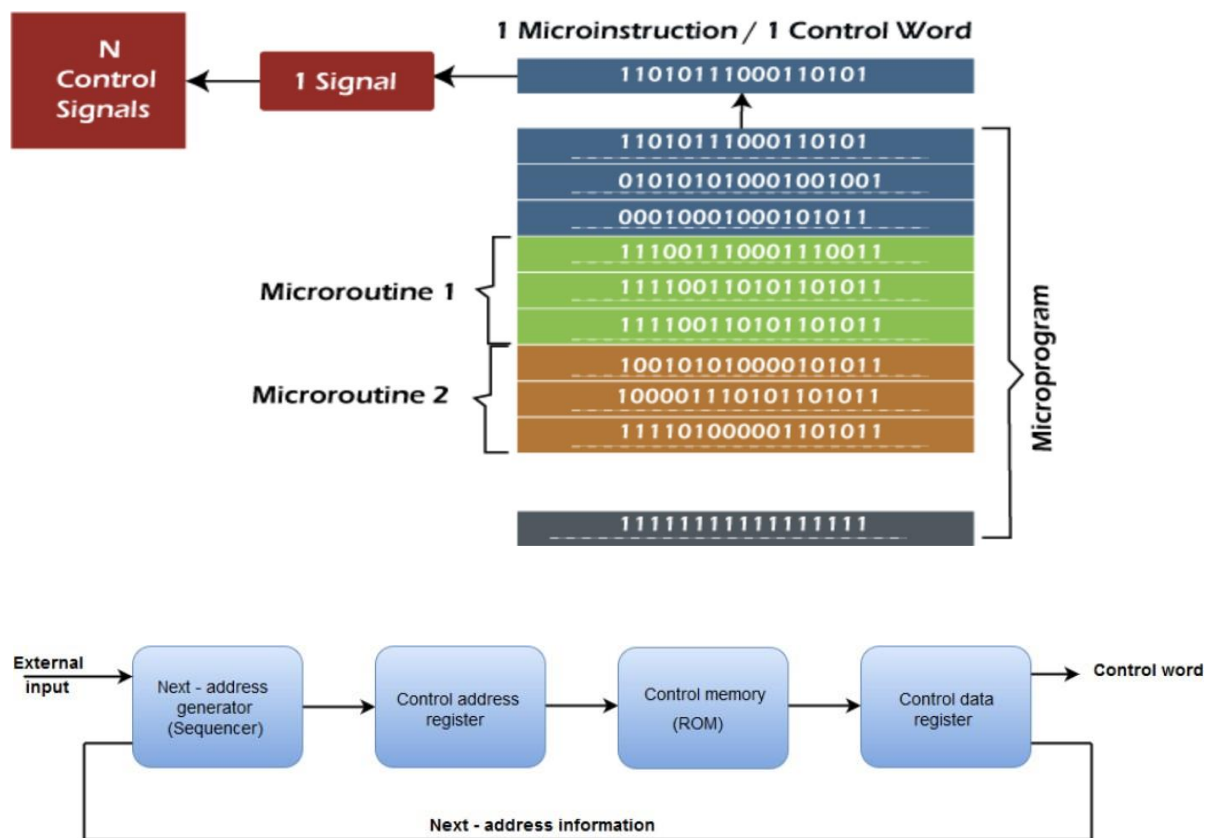
instruction decoder sets the corresponding Instruction signal INS_i to 1.

- **Step Counter** - specifies the current step of instruction execution. It contains the signals from $T_1, \dots, T_5$. Now on the basis of the step which contains the instruction, one of the signals of a step counter will be set from T_1 to T_5 to 1.
- **Clock** - The one-clock cycle of the clock will be completed for each step. For example, suppose that if the step counter sets T_3 to 1, then after completing one clock cycle, the step counter will set T_4 to 1.
- **Counter Enable** will "disable" the Step Counter so that it will stop till current step of execution is complete, then increment to the next step signal.
- **Condition Signals** - There are various conditions in which the signals are generated with the help of control signals that can be less than, greater than, less than equal, greater than equal, and many more.
- The **external input** is the last one. It is used to tell the Control Signal Generator about the interrupts, which will affect the execution of an instruction.

Microprogrammed Control Unit

- A control unit whose binary control values are saved as words in memory is called a microprogrammed control unit.

1. **Control Word:** A control word is a word whose individual bits represent various control signals.
2. **Micro-routine:** A sequence of control words corresponding to the control sequence of a machine instruction constitutes the micro-routine for that instruction.
3. **Micro-instruction:** Individual control words in this micro-routine are referred to as microinstructions.
4. **Micro-program:** A sequence of micro-instructions is called a micro-program, which is stored in a ROM or RAM called a Control Memory (CM).
5. **Control Store:** the micro-routines for all instructions in the instruction set of a computer are stored in a special memory called the Control Store.



- The Control memory address register specifies the address of the micro-instruction.
- The Control memory is assumed to be a ROM, within which all control information is permanently stored.
- The control register holds the microinstruction fetched from the memory.
- The micro-instruction contains a control word that specifies one or more micro-operations for the data processor.
- While the micro-operations are being executed, the next address is computed in the next address generator circuit and then transferred into the control address register to read the next microinstruction.
- The next address generator is often referred to as a micro-program sequencer, as it determines the address sequence that is read from control memory.

Comparison		
Attributes	Hardwired Control	Microprogramming Control
Speed	Fast	Slow
Cost of Implementation	More	Cheaper
Flexibility	Difficult to modify	Flexible
Ability to handle complex instruction	Difficult	Easier
Decoding	Complex	Easy
Application	RISC	CISC
Instruction Set Size	Small	Large
Control Memory	Absent	Present

Pipelining

- Pipelining is an implementation technique in which multiple instructions are overlapped in execution.
- **Pipelining** is a process of arrangement of hardware elements of the CPU such that its overall performance is increased.
- Simultaneous execution of more than one instruction takes place in a pipelined processor.

Real Life Example - Explanation refer my video

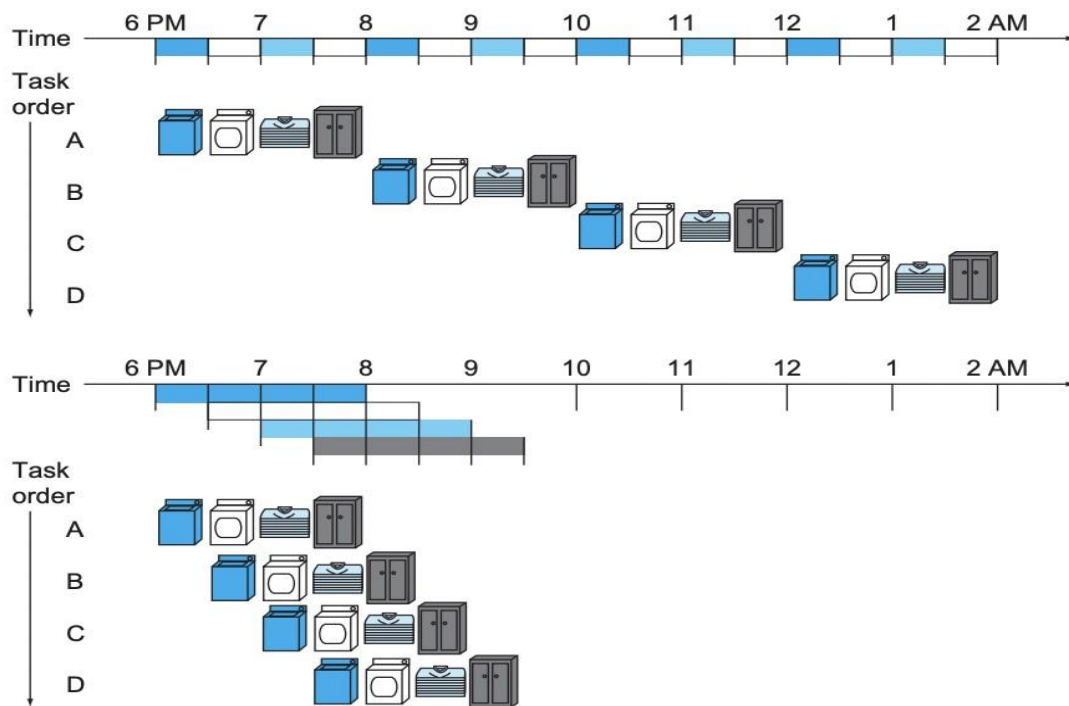
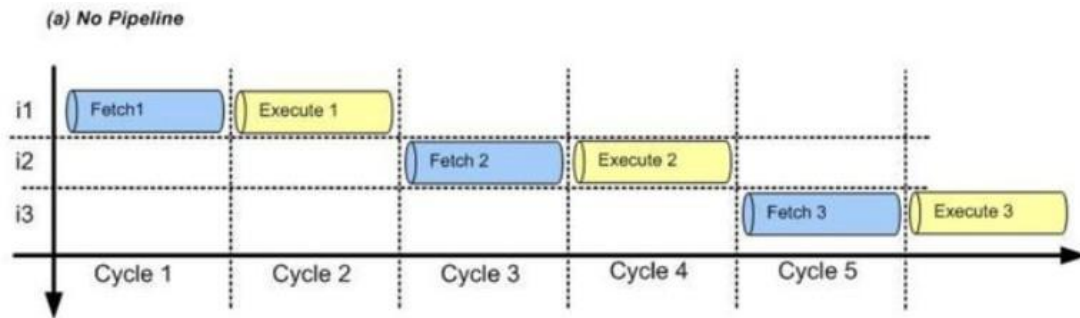


FIGURE 4.25 The laundry analogy for pipelining. Ann, Brian, Cathy, and Don each have dirty clothes to be washed, dried, folded, and put away. The washer, dryer, “folder,” and “storer” each take 30 minutes for their task. Sequential laundry takes 8 hours for 4 loads of wash, while pipelined laundry takes just 3.5 hours. We show the pipeline stage of different loads over time by showing copies of the four resources on this two-dimensional time line, but we really have just one of each resource.

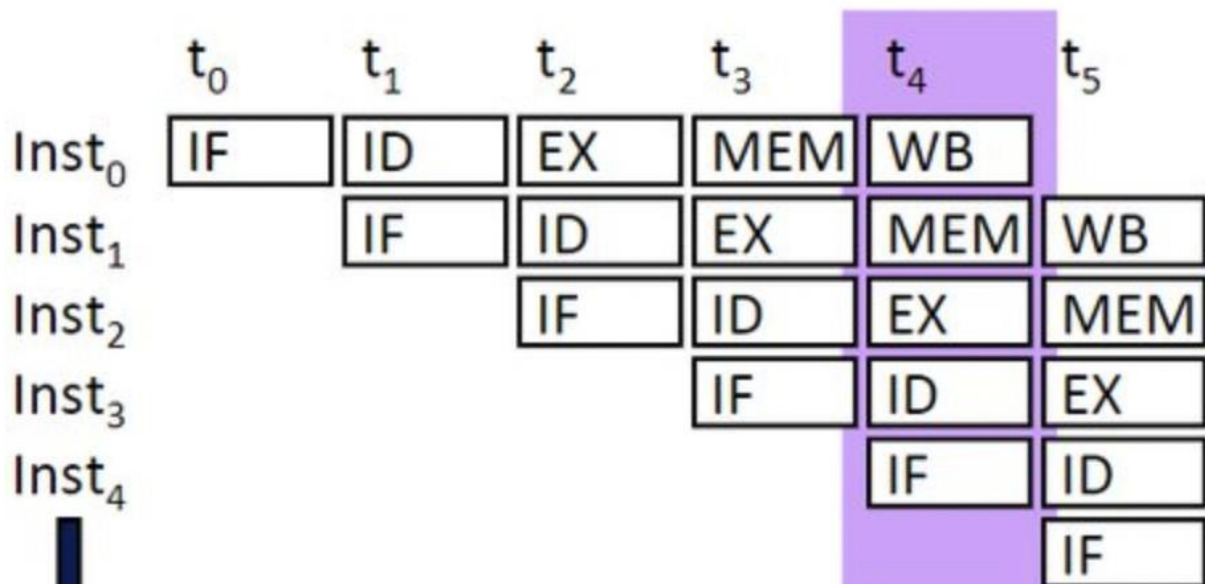
Design of a basic pipeline

- In a pipelined processor, a pipeline has two ends, the input end and the output end. Between these ends, there are multiple stages/segments such that the output of one stage is connected to the input of the next stage and each stage performs a specific operation.
- Interface registers are used to hold the intermediate output between two stages. These interface registers are also called latch or buffer.
- All the stages in the pipeline along with the interface registers are controlled by a common clock.

Diagrammatic Representation of No-Pipeline vs Pipeline



Pipelined Version



Main points:

- Instruction pipelining is a technique that implements a form of parallelism called as instruction level parallelism within a single processor.
- Multiple instructions are executed parallelly.
- **Staging:**
 - The hardware of the CPU is split up into several functional units.
 - Each functional unit performs a dedicated task.
 - The number of functional units may vary from processor to processor.
 - These functional units are called as stages of the pipeline.
 - Control unit manages all the stages using control signals.
 - There is a register associated with each stage that holds the data.
 - There is a global clock that synchronizes the working of all the stages.
 - At the beginning of each clock cycle, each stage takes the input from its register.
 - Each stage then processes the data and feed its output to the register of the next stage.



Advantages vs Drawbacks

Advantages of Pipelining

- Instruction throughput increases.
- Increase in the number of pipeline stages increases the number of instructions executed simultaneously.
- Faster ALU can be designed when pipelining is used.
- Pipelined CPU's works at higher clock frequencies than the RAM.
- Pipelining increases the overall performance of the CPU.

Disadvantages of Pipelining

- Designing of the pipelined processor is complex.
- Instruction latency increases in pipelined processors.
- The throughput of a pipelined processor is difficult to predict.
- The longer the pipeline, worse the problem of hazard for branch instructions.

Hazards

There are situations in pipelining when the next instruction cannot execute in the following clock cycle. These events are called hazards, and there are three different types.

1) Structural Hazard

2) Data Hazard

3) Control Hazard



1. Structural Hazard

- ❖ It means that the hardware cannot support the combination of instructions that we want to execute in the same clock cycle.
- ❖ This dependency arises due to the resource conflict in the pipeline. A resource conflict is a situation when more than one instruction tries to access the same resource in the same cycle. A resource can be a register, memory, or ALU.

Instruction / Cycle	1	2	3	4	5
I ₁	IF(Mem)	ID	EX	Mem	
I ₂		IF(Mem)	ID	EX	
I ₃			IF(Mem)	ID	EX
I ₄				IF(Mem)	ID

- In the above scenario, in cycle 4, instructions I₁ and I₄ are trying to access same resource (Memory) which introduces a resource conflict.
- To avoid this problem, we have to keep the instruction on wait until the required resource (memory in our case) becomes available. This wait will introduce stalls in the pipeline as shown below:

Cycle	1	2	3	4	5	6	7	8
I ₁	IF(Mem)	ID	EX	Mem	WB			
I ₂		IF(Mem)	ID	EX	Mem	WB		
I ₃			IF(Mem)	ID	EX	Mem	WB	
I ₄				-	-	-	IF(Mem)	

2. Data Hazard

Data hazards occur when instructions that exhibit data dependence, modify data in different stages of a pipeline.

I1 : ADD R1, R2, R3

I2 : SUB R4, R1, R2

Instruction / Cycle	1	2	3	4
I ₁	IF	ID	EX	DM
I ₂		IF	ID(Old value)	EX

- When the above instructions are executed in a pipelined processor, then data dependency condition will occur, which means that I2 tries to read the data before I1 writes it, therefore, I2 incorrectly gets the old value from I1.
- To minimize data dependency stalls in the pipeline, **operand forwarding** is used.

Operand Forwarding : In operand forwarding, we use the interface registers present between the stages to hold intermediate output so that dependent instruction can access new value from the interface register directly.

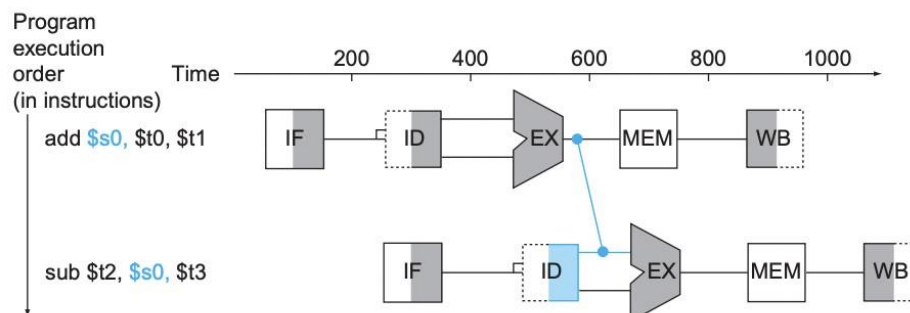


FIGURE 4.29 Graphical representation of forwarding. The connection shows the forwarding path from the output of the EX stage of add to the input of the EX stage for sub, replacing the value from register \$s0 read in the second stage of sub.

3. Control Hazard



- ❖ This type of dependency occurs during the transfer of control instructions such as BRANCH, CALL, JMP, etc.
- ❖ On many instruction architectures, the processor will not know the target address of these instructions when it needs to insert the new instruction into the pipeline.
- ❖ Due to this, unwanted instructions are fed to the pipeline.
- ❖ Eg. 100: I₁

101: I₂ (JMP 250) (Jump address known after ID stage only)

102: I₃

250: B1₁

Expected output: I₁ -> I₂ -> B1₁

Instruction/ Cycle	1	2	3	4	5	6
I ₁	IF	ID	EX	MEM	WB	
I ₂		IF	ID (PC:250)	EX	Mem	WB
I ₃			IF	ID	EX	Mem
B1 ₁				IF	ID	EX

The output which we get I₁ -> I₂ -> I₃ -> B1₁

So, the output sequence is not equal to the expected output, that means the pipeline is not implemented correctly.

1. **Using Stalls** - To correct the above problem we need to stop the Instruction fetch until we get target address of branch instruction. This can be implemented by introducing delay slot until we get the target address.

Instruction/ Cycle	1	2	3	4	5	6
I ₁	IF	ID	EX	MEM	WB	
I ₂		IF	ID (PC:250)	EX	Mem	WB
Delay	-	-	-	-	-	-
B1 ₁				IF	ID	EX

Output Sequence: I₁ -> I₂ -> Delay (Stall) -> B1₁

2. Branch Prediction - There are 2 different types of prediction

1. Static
 - a. In this strategy branch can be predicted based on branch code types statically. This means that the probability of branch with respect to a particular branch type is used to predict the branch.
 - b. This branch strategy may not produce accurate results every time. One improvement over branch stalling is to predict that the branch will not be taken and thus continue execution down the sequential instruction stream.
2. Dynamic

- a. This strategy uses recent branch history during program execution to predict whether or not the branch will be taken next time when it occurs. It uses recent branch information to predict the next branch. This technique is called dynamic branch prediction.
- b. A branch prediction buffer or branch history table is a small memory indexed by the lower portion of the address of the branch instruction. The memory contains a bit that says whether the branch was recently taken or not.

3. Delayed Branching

- 1) The slot directly after a delayed branch instruction, which in the MIPS architecture is filled by an instruction that does not affect the branch.
- 2) An instruction that always executes after the branch in the branch delay slot.